

Task 1

```
1  import math
2
3  hr_resolution = int(input("Enter horizontal resolution (pixels): "));
4  vr_resolution = int(input("Enter vertical resolution (pixels): "));
5  physical_diagonal_size = float(input("Enter physical diagonal size (inches): "))
6  total_pixel_count = hr_resolution * vr_resolution;
7  DPI = math.sqrt(hr_resolution ** 2 + vr_resolution ** 2) / physical_diagonal_size
8
9  def get_ratio(a,b):
10     divisor = math.gcd(a,b)
11
12     simplified_a = a // divisor
13     simplified_b = b // divisor
14
15     return f"{simplified_a}:{simplified_b}"
16
17
18  print("--- DISPLAY METRICS ANALYSIS ---");
19  print(f"Total Pixel Count : {total_pixel_count} pixels");
20  print("Aspect Ratio : ", get_ratio(hr_resolution, vr_resolution))
21  print(f"Calculated DPI : {round(DPI, 2)} DPI")
22  if DPI < 100:
23     print("Density Category : Low Density (Standard Monitor)")
24  elif DPI <= 200:
25     print("Density Category : Medium Density (HD Display)")
26  else:
27     print("Density Category : High Density (Retina / Mobile)")
```

Output 1 (1920x1080, 24")

```
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24
--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2073600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
```

Output 2 (1170x2532, 6.1")

```
• Enter horizontal resolution (pixels): 1170
  Enter vertical resolution (pixels): 2532
  Enter physical diagonal size (inches): 6.1
  --- DISPLAY METRICS ANALYSIS ---
  Total Pixel Count : 2962440 pixels
  Aspect Ratio : 195:422
  Calculated DPI : 457.25 DPI
  Density Category : High Density (Retina / Mobile)
```

Task 2

```
1  import numpy as np
2
3  image = np.zeros((300, 400, 3), dtype=np.uint8)
4
5  image[0:150, 0:200] = [255, 0, 0]
6  image[0:150, 200:400] = [0, 255, 0]
7  image[150:300, 0:200] = [0, 0, 255]
8  image[150:300, 200:400] = [255, 255, 255]
9
10 print("--- SYNTHETIC MATRIX METRICS ---")
11 print("Array Shape (H, W, C) :", image.shape)
12 print("Data Type           :", image.dtype)
13 print("Total Elements      :", f"{image.size:,}", "values")
14 print("Memory Footprint    :", f"{image.nbytes:,}", "bytes", f"({image.nbytes / 1024:.2f})", "KB")
```

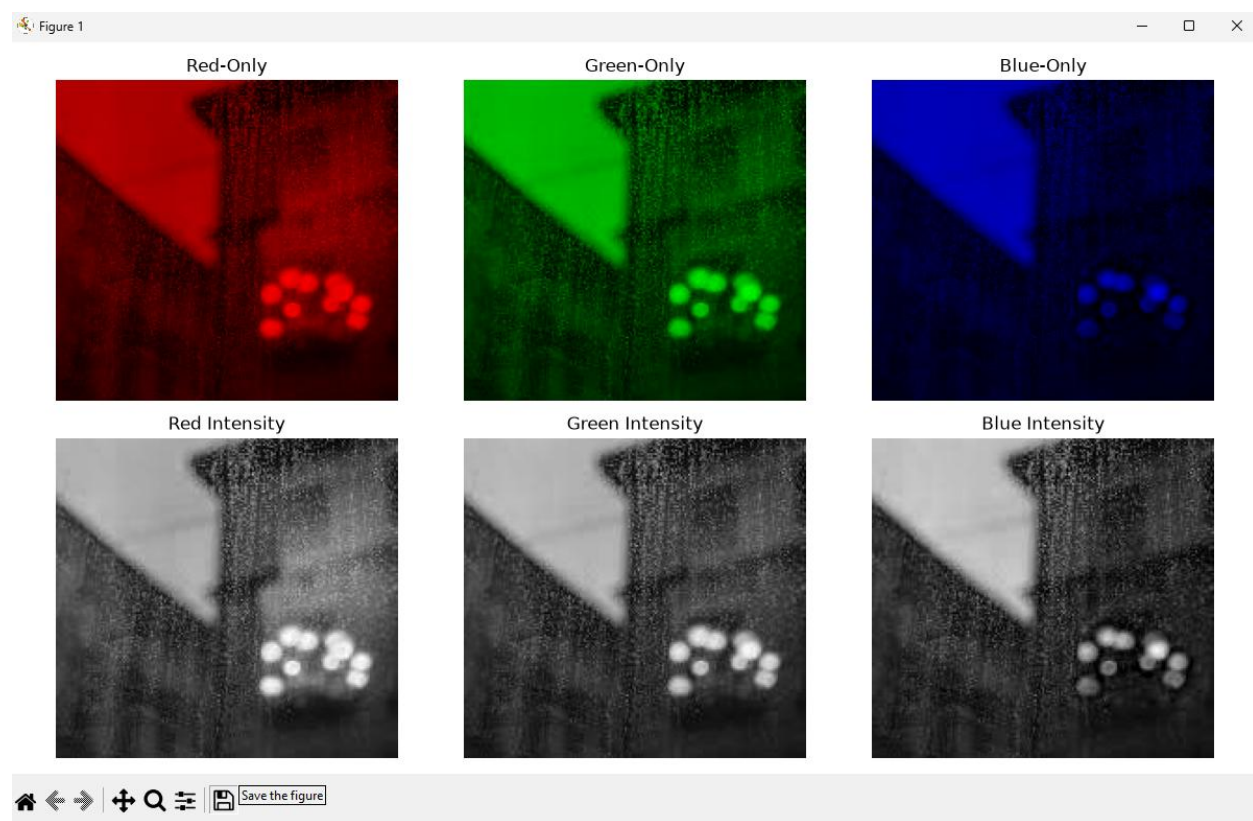
Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type           : uint8
Total Elements      : 360,000 values
Memory Footprint    : 360,000 bytes (351.56) KB
```

Task 3

```
1  import numpy as np
2  from PIL import Image
3  import matplotlib.pyplot as plt
4
5  image = Image.open("sample.jpg").convert("RGB")
6
7  img = np.array(image)
8
9  red = img[:, :, 0]
10 green = img[:, :, 1]
11 blue = img[:, :, 2]
12
13 red_only = np.zeros_like(img)
14 green_only = np.zeros_like(img)
15 blue_only = np.zeros_like(img)
16
17 red_only[:, :, 0] = red
18 green_only[:, :, 1] = green
19 blue_only[:, :, 2] = blue
20
21 print("--- CHANNEL EXTRACTION SUMMARY ---")
22 print("Original Image Shape :", img.shape)
23
24 print(
25     f"Red Channel 2D Shape : {red.shape} | "
26     f"Mean Intensity: {red.mean():.2f}"
27 )
28
29 print(
30     f"Green Channel 2D Shape: {green.shape} | "
31     f"Mean Intensity: {green.mean():.2f}"
32 )
33
34 print(
35     f"Blue Channel 2D Shape : {blue.shape} | "
36     f"Mean Intensity: {blue.mean():.2f}"
37 )
38
39 plt.figure(figsize=(12, 7))
40
41 plt.subplot(2, 3, 1)
42 plt.imshow(red_only)
43 plt.title("Red-Only")
44 plt.axis("off")
45
46 plt.subplot(2, 3, 2)
47 plt.imshow(green_only)
48 plt.title("Green-Only")
49 plt.axis("off")
50
51 plt.subplot(2, 3, 3)
52 plt.imshow(blue_only)
53 plt.title("Blue-Only")
54 plt.axis("off")
55
56 plt.subplot(2, 3, 4)
57 plt.imshow(red, cmap="gray")
58 plt.title("Red Intensity")
59 plt.axis("off")
60
61 plt.subplot(2, 3, 5)
62 plt.imshow(green, cmap="gray")
63 plt.title("Green Intensity")
64 plt.axis("off")
65
66 plt.subplot(2, 3, 6)
67 plt.imshow(blue, cmap="gray")
68 plt.title("Blue Intensity")
69 plt.axis("off")
70
71 plt.tight_layout()
72 plt.show()
73
74 print("Display Window : Matplotlib 2x3 Subplot Grid Rendered")
```

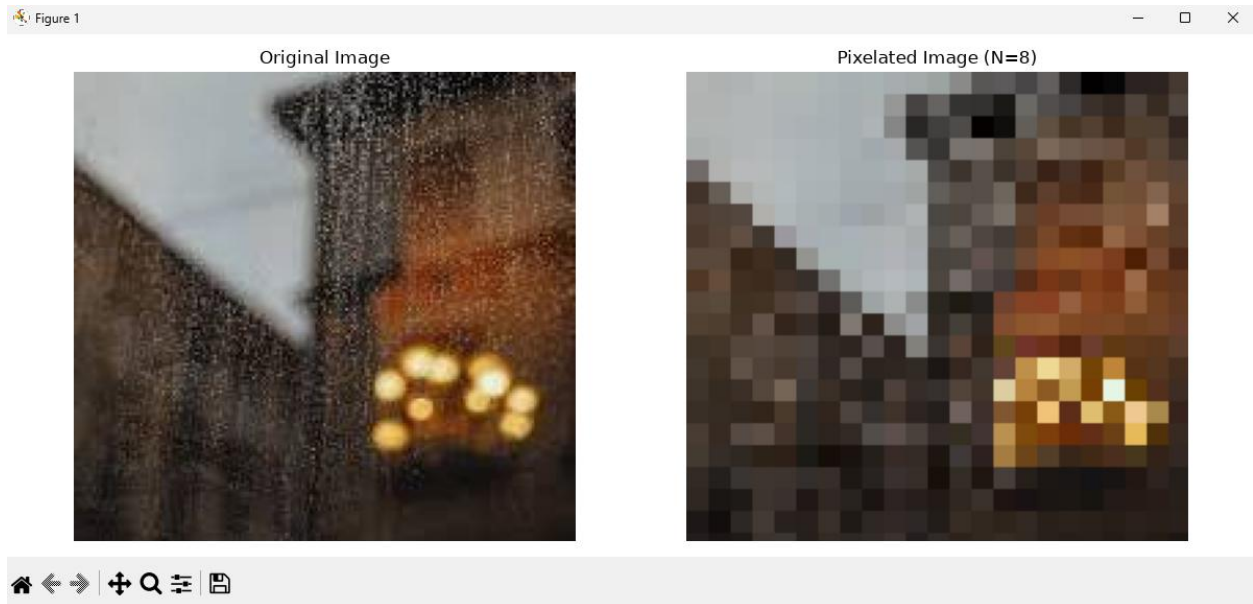
Output



Task 4

```
1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 # Load the image
6 image = Image.open("sample.jpg").convert("RGB")
7 img = np.array(image)
8
9 # Downsampling factor
10 N = 8
11
12 # Original dimensions and memory
13 original_shape = img.shape
14 original_memory = img.nbytes
15
16 # 1. Downsample using NumPy striding
17 downsampled = img[::N, ::N, :]
18
19 # 2. Re-expand using np.repeat()
20 expanded = np.repeat(
21     np.repeat(downsampled, N, axis=0),
22     N,
23     axis=1
24 )
25
26 # Make sure expanded image has the same dimensions as original
27 expanded = expanded[:img.shape[0], :img.shape[1], :]
28
29 # 3. Calculate reductions
30
31 # Reduction per spatial axis
32 height_reduction = (1 - downsampled.shape[0] / img.shape[0]) * 100
33 width_reduction = (1 - downsampled.shape[1] / img.shape[1]) * 100
34
35 # Memory reduction
36 memory_savings = (
37     1 - downsampled.nbytes / original_memory
38 ) * 100
39
40 # Print results
41 print("--- DOWNSAMPLING ANALYSIS (N = 8) ---")
42
43 print(
44     f"Original Shape      : {original_shape} | "
45     f"Memory: {original_memory:,} bytes"
46 )
47
48 print(
49     f"Downsampled Shape   : {downsampled.shape} | "
50     f"Memory: {downsampled.nbytes:,} bytes"
51 )
52
53 print(
54     f"Re-expanded Shape    : {expanded.shape} | "
55     f"Visual: Blocky Pixelation"
56 )
57
58 print(
59     f"Dimension Reduction : {height_reduction:.2f}% "
60     f"reduction per axis"
61 )
62
63 print(
64     f"Memory Savings       : {memory_savings:.2f}% data reduction"
65 )
66
67 # Display original and pixelated images
68 plt.figure(figsize=(12, 5))
69
70 plt.subplot(1, 2, 1)
71 plt.imshow(img)
72 plt.title("Original Image")
73 plt.axis("off")
74
75 plt.subplot(1, 2, 2)
76 plt.imshow(expanded)
77 plt.title("Pixelated Image (N=8)")
78 plt.axis("off")
79
80 plt.tight_layout()
81 plt.show()
```

Output



Q: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Ans: Adding an Alpha channel increases memory consumption by approximately 33.33% because RGB stores 3 bytes per pixel (Red, Green, Blue), while RGBA stores 4 bytes per pixel by adding the Alpha/transparency channel. The additional 1 byte represents $\frac{1}{3}$ of the original RGB memory, resulting in a 33.33% increase.